

Verified, Fast, Deep? A Reasoning LLM, a Zero-Shot Classifier and Tabular Foundation Models in One Data-Analyst Agent

Yarin Benizri

Hebrew University of Jerusalem · open models only, on a Slurm cluster with H200 GPUs · code: `excel_magic/vfd-agent` · draft, 3 October 2026 · companion papers: *Open TabFM-Auto, Which Layer Is the Embedding?*

ABSTRACT. Company data-analyst agents are usually one reasoning LLM with SQL tools. We ask whether two other kinds of open model can make such an agent better: a small zero-shot classifier (GLiClass) that checks the agent's steps quickly, and frozen tabular and relational foundation models (Kumo Tabular, Kumo Relational, TabPFN) that read all the rows, not only the rows that fit in the prompt. We give each model one job: the LLM plans and writes SQL, the classifier verifies (“verified, fast”), and the tabular models predict and explain (“deep”). We test this on four benchmarks with open models only. (1) As a verifier of SQL candidates on BIRD, zero-shot GLiClass reaches AUROC 0.685 at 5 ms per candidate, better than the generator's own signals and self-consistency (0.651) but below an LLM judge (0.786, 12× slower); added to the free signals it lifts a stacked verifier from 0.676 to 0.725. It is also the best of five schema linkers. (2) End to end, these gains do not turn into accuracy: no small-model configuration beats plain self-consistency (0.705 vs 0.693 greedy), and schema linking costs -0.026. (3) A guarded auto-research loop rejected 12 of 13 proposed changes; the one it kept raised held-out accuracy from 0.680 to 0.720 (se 0.018). (4) The tabular models add a capability the LLM does not have: on 8 RelBench prediction questions, giving the agent Kumo Relational and Kumo Tabular as tools raises mean AUROC from 0.577 (SQL only) to 0.732. (5) On InsightBench, which scores descriptive insights by recall, the extra tools and the verified insight ledger lower the score (0.304 → 0.230), and the same holds when the tools are driven by the pi coding agent (0.273 → 0.210). The models help where their job is the bottleneck, and cost steps where it is not.

1. Introduction

A “BI analyst” agent answers business questions over company databases. The usual design is one large reasoning LLM that reads the schema, writes SQL, reads the result and writes an answer. This design has three known limits. It is hard to know when an answer is wrong. Checking with a second LLM is slow and costly. And the LLM sees only what fits in its prompt: it can count and average, but it cannot fit a model on a million rows to say what drives an outcome or who will churn.

Two kinds of small, open models address these limits on paper. Zero-shot classifiers such as GLiNER and GLiClass score any text against any label in a few milliseconds; they could check each step of the agent. Tabular foundation models (TabPFN, Kumo Tabular) and relational ones (Kumo Relational) predict on new tables with no training; they could give the agent a view of all the data. We ask whether this works in practice, and where.

Contributions. (i) An open system that gives each model one job (Section 3), as a pipeline harness for SQL, as an analyst tool loop, and as a tool server that any coding agent (here pi) can call. (ii) A verifier study: how well a zero-shot

classifier tells correct SQL from wrong SQL, against free signals, an NLI model and an LLM judge, and how well it links questions to columns. (iii) End-to-end ablations on BIRD, InsightBench and RelBench, and a text-feature study on 18 tables with text. (iv) A guarded auto-research loop that tunes the harness without overfitting. (v) Negative results, reported as such.

2. Related work

Text-to-SQL. BIRD (Li et al., 2023) is the standard benchmark; many of its gold queries are wrong, and we use the 498-question mini-dev set with the Arcwise-Plat corrections. Strong systems sample many candidates and pick one, by self-consistency (Wang et al., 2023) or by a trained or LLM selector; schema linking (e.g. CHESS, Talaei et al., 2024) shortens the prompt.

Small zero-shot models. GLiNER (Zaratiana et al., 2024) extracts entities of any type with a small bidirectional encoder; GLiClass applies the same idea to classification. NLI cross-encoders are an older zero-shot classifier. LLM-as-judge (Zheng et al., 2023) is the strong but slow verifier.

Tabular and relational foundation models. TabPFN (Hollmann et al., 2025) predicts in

context on small tables; Kumo Tabular S/L read larger contexts; relational models such as KumoRFM (Fey et al., 2025) and Kumo Relational predict over a database graph. RelBench (Robinson et al., 2024) gives prediction tasks over real databases with official splits.

Analyst agents and auto-research. InsightBench (Sahu et al., 2025) scores the insights an agent finds in a table against planted ones. Auto-research loops (Karpathy, 2026) let an LLM propose and test changes to a system; our loop adds the overfitting guards of our companion paper *Open TabFM-Auto*.

3. System

Roles. The *reasoning LLM* plans, writes SQL and writes the answer. The *zero-shot classifier* (GLiClass large v3, zero-shot, no fine-tuning) does three cheap jobs: it links a question to schema columns, it scores a (question, SQL, result) triple against hypotheses such as “the query answers the question” and an error taxonomy, and it labels free-text columns. The *tabular and relational models* run behind a DEEP tool with seven operations: predict, drivers (permutation importance), what-if (partial dependence), anomalies (out-of-fold surprise), drift (two-sample classifier), similar entities and a hypothesis test (paired repeated splits). Each DEEP answer is checked against a gradient-boosting model when the data are large or temporal.

SQL harness. A fixed pipeline: link (top-k columns plus keys) → write → triage (if the classifier’s score is below a threshold, one revision with the predicted error labels) → execute → verify (greedy, self-consistency, a stacked logistic regression of cheap signals, or a cascade that sends the uncertain band to an LLM judge). Configurations A-F add one stage at a time (Table 1).

Analyst loop. An OpenAI-style tool loop over DuckDB. D has `sql` and `record_insight`; E adds the DEEP tools; F adds `text_labels` and a *verified ledger*: an insight is accepted only if its evidence SQL runs again, every number in the sentence appears in that result (with rounding and ratio rules), and GLiClass finds the sentence entailed by the result.

Prediction questions. For RelBench, the agent gets the database cut at the test time, a `train_labels` and a `test_rows` table, and must submit a score per test row. D has SQL only; E adds `deep_fit_predict` (Kumo Tabular-L on features the agent writes in SQL); R adds `relational_predict` (Kumo Relational’s graph layer

with a linear probe, the best layer found in our companion paper); ER has both.

Other harnesses. The same tools run as a local HTTP service with a `vfd <tool>` shell command, so the pi coding agent (used with GLM at the author’s company) can call them unchanged.

Table 1. Configurations. Each adds one thing to the one above it.

id	benchmark	what is added
A	BIRD	LLM alone, greedy, full schema
B	BIRD	GLiClass schema linking
C	BIRD	GLiClass triage + one revision
D	BIRD	8 candidates + cheap verifier stack
F	BIRD	LLM judge on the uncertain band
SC	BIRD	baseline: 8 candidates, majority vote
D/E/F	InsightBench	SQL / + DEEP / + labels + verified ledger
D/E/R/ER	RelBench	SQL / + Kumo Tabular / + Kumo Relational / both

4. Setup

Models. All open. Generator and analyst: Qwen3-Coder-30B-A3B-Instruct (vLLM). Judge and auto-research researcher: GLM-4.5-Air-FP8, a different model family, thinking off. GLiClass large v3.0; NLI: DeBERTa-v3-large zero-shot v2.0; linkers: bge-small, bge-reranker-v2-m3, GLiNER bi-base v2.0, lexical overlap. Tabular: Kumo Tabular-L and S, TabPFN-2.5, LightGBM; relational: Kumo Relational. One H200 per job.

Benchmarks. BIRD mini-dev with the Arcwise-Plat corrections (498 questions; BIRD set-equality of results). AutoGluon multimodal text+tabular benchmark (Shi et al., 2021), 18 datasets, official splits (train ≤ 3,000, test ≤ 2,000 rows). InsightBench, 100 tables, scored by an open judge (GLM-4.5-Air, G-Eval-style best match per planted insight) and ROUGE-1. RelBench, 8 entity classification tasks, official evaluator. Every comparison is paired over the same questions, tables or tasks; we report the standard error of the paired difference.

5. Results

5.1 Verified and fast: the classifier as a verifier

Table 2. Which signal tells a correct SQL candidate from a wrong one? BIRD Arcwise-Plat, 497 questions, 4,473 candidates (1 greedy + 8 sampled). Greedy accuracy 0.692, oracle (any candidate correct) 0.746. Stacks: logistic regression, 2-fold by question.

signal	AUROC	within-question AUROC	best-of-N acc.	ms / item
query executes	0.556	0.631	0.706	0
result not empty	0.600	0.657	0.706	0
generator log-prob	0.579	0.512	0.692	0
self-consistency (vote share)	0.651	0.721	0.706	0
GLiClass, zero-shot	0.685	0.695	0.712	5
NLI cross-encoder	0.656	0.733	0.720	5
LLM judge (other family)	0.786	0.748	0.714	64
stack: no-model signals	0.676	0.725	0.706	
stack: + GLiClass	0.725	0.766	0.712	
stack: + NLI + judge	0.791	0.762	0.714	

Table 2 asks one question per signal: does a higher score mean a correct candidate? Signals that come for free are weak: whether the query

Table 4. The SQL harness, end to end. BIRD Arcwise-Plat, 498 questions, Qwen3-Coder-30B-A3B. Paired difference against A, standard error over questions.

configuration	accuracy	vs A (se)	schema chars	LLM calls	s / question
A: LLM alone, greedy, full schema	0.693		4,994	1.00	1.5
B: + GLiClass schema linking (top 20 + keys)	0.667	-0.026 (0.016)	2,254	1.00	2.6
C: + GLiClass triage, one revision	0.677	-0.016 (0.016)	2,254	1.03	2.7
D: + 8 candidates, cheap verifier stack	0.705	+0.012 (0.016)	2,254	2.27	9.3
F: D + LLM judge on the uncertain band	0.703	+0.010 (0.015)	2,254	2.26 + 1.5	9.0
SC: 8 candidates, majority vote, no small models	0.705	+0.012 (0.009)	4,994	2.00	4.0

Table 4 puts the stages together. Linking halves the prompt (4,994 $\rightarrow$ 2,254 characters) but loses accuracy (-0.026, se 0.016): even the best linker leaves out at least one needed column in 38% of questions, and one missing column breaks the query. Triage wins back one point. With 8

runs (0.556) and the generator's log-probability (0.579); self-consistency is better (0.651). Zero-shot GLiClass, with no training on SQL, reaches 0.685 at 5 ms per candidate, above the NLI cross-encoder (0.656). The LLM judge is best (0.786) but 12 $\times$ slower per call. The signals are complementary: GLiClass lifts the stack of free signals from 0.676 to 0.725; adding the judge gives 0.791.

The catch is in the last column. Picking the best of 9 candidates moves accuracy only from 0.692 to at most 0.746 (the oracle), and no verifier gets close to the oracle: on a corrected benchmark with a strong generator, most questions are either always right or always wrong.

Table 3. Schema linking (497 questions): share of the gold SQL's columns in the linker's top k; all@20: questions with every gold column in the top 20; ms per question.

linker	@5	@10	@20	all@20	ms
GLiClass large v3	0.648	0.788	0.877	0.619	50
lexical overlap	0.576	0.706	0.811	0.468	0
bge-reranker-v2-m3	0.541	0.687	0.802	0.520	105
bge-small embedding	0.503	0.645	0.774	0.472	15
GLiNER bi-encoder	0.240	0.350	0.530	0.213	49

For schema linking (Table 3), GLiClass ranks the needed columns best: 0.788 of them are in its top 10, against 0.706 for lexical overlap; the GLiNER bi-encoder, built for spans and not for column names, is weak (0.350).

5.2 End to end on BIRD

candidates, the stacked verifier (D, 0.705) and the judge cascade (F, 0.703) only match plain self-consistency (SC, 0.705), which needs no extra model. The verifier ranks well (Section 5.1), but here there is little to rank.

5.3 Guarded auto-research

Table 5. The guarded auto-research loop. Researcher GLM-4.5-Air; split fixed before the loop: search 298, accept 75, held-out 125 questions. Keep rule: search gain > 1 se and accept gain ≥ 0 .

#	change	search gain (se)	accept gain	decision
1	verifier=self_consistency	-0.010 (0.007)	+0.027	rejected
2	link=gliclass	-0.020 (0.017)	+0.000	rejected
3	n=4	-0.010 (0.006)	+0.027	rejected
4	verifier=stack	-0.010 (0.007)	+0.027	rejected
5	trriage=True	+0.013 (0.009)	+0.013	kept
6	trriage_threshold=0.3	-0.003 (0.006)	+0.000	rejected
7	link_k=10	-0.007 (0.007)	-0.013	rejected
8	link_k=40	-0.007 (0.007)	+0.000	rejected
9	link=rerank	-0.067 (0.020)	-0.013	rejected
10	n=8	+0.003 (0.006)	-0.027	rejected
11	n=16	-0.003 (0.006)	+0.000	rejected
12	trriage_threshold=0.7	+0.000 (0.008)	+0.000	rejected
13	trriage=False	-0.013 (0.009)	-0.013	rejected

The researcher LLM proposed one change at a time from a fixed space (linker, top k, triage and its threshold, number of candidates, verifier). Of 13 changes, it kept 1: triage=True. On the held-out questions, scored once at the end, accuracy went from 0.680 to 0.720 (paired +0.040, se 0.018). The guards did their job: three rejected changes had an accept-slice gain of +0.027 and would have been kept by a loop that looked only

at one split. Two changes that do nothing (top k with linking off) still moved the search score by up to 0.007, because vLLM greedy decoding is not bit-exact under batching; this is the noise floor any such loop must beat. The kept change is small and depends on the rest of the configuration: in Table 4, triage with linking on was below A.

5.4 The classifier as a featurizer

Table 6. Text columns as features. Mean change of the official test metric (accuracy, AUROC or R^2) against numeric and categorical columns only; in brackets, datasets that improve. 18 datasets of the AutoGluon multimodal benchmark.

model	tf-idf + SVD	GLiClass labels	embedding	labels + embedding	labels + emb. beat emb.
Kumo Tabular-L	+0.157 (15/18)	+0.155 (17/18)	+0.184 (16/18)	+0.190 (16/18)	15/18
LightGBM	+0.120 (16/18)	+0.128 (17/18)	+0.145 (16/18)	+0.159 (16/18)	18/18
TabPFN-2.5	+0.125 (14/17)	+0.126 (14/17)	+0.160 (16/17)	+0.159 (14/17)	10/17

When a table has text columns, the LLM can propose label sets and GLiClass can turn each text into label scores, which a tabular model reads as features. Table 6 shows that labels help every model on most datasets, but less than a dense sentence embedding. They are complementary: labels plus embedding beat the embedding alone on 15 of 18 datasets for Kumo Tabular-L and 18 of 18 for LightGBM. Labels have one extra value the table does not show: they are readable (“mentions a refund”), so the agent can talk about them.

5.5 Deep: prediction questions over databases

Table 8. Prediction questions over RelBench databases (“for each user, how likely is ...”). Official test AUROC, mean of 2 episodes per cell; the agent writes features in SQL and calls the models as tools.

task	D: LLM + SQL	E: + Kumo Tabular-L	R: + Kumo Relational	ER: both
avito clicks	0.584	0.661	0.624	0.627
avito visits	0.634	0.641	0.653	0.662
event ignore	0.500	0.567	0.877	0.840
event repeat	0.556	0.711	0.794	0.754
f1 dnf	0.506	0.792	0.713	0.744
f1 top3	0.749	0.784	0.863	0.863
hm churn	0.595	0.681	0.674	0.674
trial outcome	0.491	0.591	0.690	0.690
mean	0.577	0.679	0.736	0.732
tool calls / episode	14.3	13.3	9.9	12.8
seconds / episode	82	63	64	96

Table 7. InsightBench, 100 tables, analyst Qwen3-Coder-30B. Insight score: open judge (GLM-4.5-Air), G-Eval-style best match of each planted insight, scaled to 0-1. Top: our tool loop. Bottom: the same tools served to the pi coding agent.

harness and tools	tables	insight score	ROUGE-1	insights	rejected	vs D / vs pi D (se)
D: LLM + SQL	100	0.304	0.245	5.6	0.0	
E: + DEEP tools (tabular FM)	100	0.266	0.235	4.8	0.0	-0.038 (0.018)
F: + text labels + verified ledger	100	0.230	0.201	3.2	2.5	-0.074 (0.022)
pi + D tools	100	0.273	0.194	6.8	0.0	
pi + E tools	100	0.269	0.191	6.7	0.0	-0.004 (0.026)
pi + F tools	100	0.210	0.161	5.2	5.8	-0.063 (0.030)

InsightBench is a negative result for our design. Its planted insights are descriptive (counts, shares and trends on tables of about 500 rows) and its score rewards recall. The DEEP tools (E, 0.266) cost steps without adding descriptive

Table 8 is where the tabular models add what the LLM cannot do alone. With SQL only (D), the agent writes a scoring rule by hand, and its mean AUROC is 0.577, near chance on several tasks. With Kumo Tabular-L fitted on its own SQL features (E) it reaches 0.679 (better than D on 8 of 8 tasks). With Kumo Relational, which reads the database graph and needs no feature SQL (R), it reaches 0.736 (better on 8 of 8); with both, 0.732 (better on 8 of 8). The agent submitted an answer in 98% of episodes. Having both tools (ER) is not better than R: in most ER episodes the agent submits the relational scores as they are. The two models are complementary per task, not on average: E beats R on f1 dnf, avito clicks and hm churn, where the agent's own features carry signal that the graph layer misses. Two tool bugs were found and fixed during this run (a query time limit, and GPU memory on the largest test tables); the affected cells were run again from scratch.

5.6 Descriptive insights (InsightBench) and the pi harness

insights, and the verified ledger (F, 0.230) rejects about 2.5 insights per table that the agent does not replace. Verification trades recall for precision, and this benchmark measures only recall.

The pi harness. The same tools, served through the `vfd` shell command, ran unchanged under the pi coding agent with the same model, which shows that the design does not depend on our loop. The scores tell the same story. pi with SQL only scores 0.273 (-0.032 against our loop, se 0.025); it records more insights per table but ends with none on 15 of 100 tables. Adding the DEEP tools changes nothing (-0.004, se 0.026). The verified ledger again costs recall (-0.063, se 0.030): it rejects 5.8 insights per table, against 2.5 in our loop, and pi does not replace them.

6. Discussion: where each model helps

- **The zero-shot classifier** is a good, fast *signal*: the best cheap verifier and the best schema linker we tested, at milliseconds per call. A good signal is not the same as a better answer: on BIRD the headroom for selection is small, and linking that drops one column breaks the query. It pays where checking is the bottleneck, for example to flag answers for a human, or with a weaker generator.
- **The tabular and relational models** give the agent a new capability, not a small gain: predictions over all rows of a database, from 0.577 to 0.732 AUROC. This is the clearest win in this paper, and the one an LLM-only agent cannot reach by writing better SQL.
- **Verification costs recall.** The verified ledger rejects unsupported numbers, which a company report needs, but a benchmark that only counts recall punishes it.
- **Tune with guards.** An auto-research loop finds small, real gains only if it is held to a fixed held-out set and a noise floor.

7. Limitations

One generator (Qwen3-Coder-30B) and one judge model; a weaker generator, or the much larger GLM model used in production, may change the verifier results. The BIRD mirrors have no column descriptions. InsightBench is scored by an open judge, not the GPT-4o judge of its paper, so our absolute scores are not

comparable to published ones; the paired differences are. RelBench results are 2 episodes per cell; vLLM decoding is not bit-exact, so single runs move by about one point. The pi harness was run once, and its tool calls were not logged, so we cannot say how often pi used the DEEP tools.

8. Conclusion

A reasoning LLM, a zero-shot classifier and tabular foundation models can work as one analyst agent, each with one job. The classifier is a fast and useful verifier signal, but on a strong text-to-SQL setup it does not beat self-consistency. The tabular and relational models are the real extension: they let the agent answer prediction questions over whole databases that SQL alone answers near chance. Code, run logs and every number in this paper are produced by the scripts in `vfd-agent`.

References

- Fey, M. et al. (2025). KumoRFM: a foundation model for in-context learning on relational data. Technical report.
- Hollmann, N. et al. (2025). Accurate predictions on small data with a tabular foundation model. *Nature*.
- Karpathy, A. (2026). autoresearch. GitHub repository.
- Li, J. et al. (2023). Can LLM already serve as a database interface? A big bench for large-scale database grounded text-to-SQLs (BIRD). *NeurIPS*.
- Robinson, J. et al. (2024). RelBench: a benchmark for deep learning on relational databases. *NeurIPS*.
- Sahu, G. et al. (2025). InsightBench: evaluating business analytics agents through multi-step insight generation. *ICLR*.
- Shi, X. et al. (2021). Benchmarking multimodal AutoML for tabular data with text fields. *NeurIPS Datasets and Benchmarks*.
- Talaei, S. et al. (2024). CHESS: contextual harnessing for efficient SQL synthesis. arXiv.
- Wang, X. et al. (2023). Self-consistency improves chain of thought reasoning in language models. *ICLR*.
- Zaratiana, U. et al. (2024). GLiNER: generalist model for named entity recognition using bidirectional transformer. *NAACL*.
- Zheng, L. et al. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *NeurIPS*.